Instalação

Para a instalação é preciso dar o `composer require spatie/laravel-activitylog`, assim as classes do pacote serão colocadas no projeto.

Após isso, adiquirir os arquivos referentes as **migrations** e as **configurações**. Isso é feito através do comando `php artisan vendor:publish –provider="<provedor>"-tag="<arquivo>"`.

NOTA: *É preciso efetuar as migrations depois de importar os arquivos externos!*

Utilizamos os seguintes parâmetros:

- Provedor: `"Spatie\Activitylog\ActivitylogServiceProvider"`.
- Tag da Migration: `"activitylog-migrations"`.
- Tag da Configuração: `"activitylog-config"`.

Na prática:

```
composer require spatie/laravel-activitylog

php artisan vendor:publish –provider="Spatie\Activitylog\ActivitylogServiceProvider"-
tag="activitylog-migrations"

php artisan vendor:publish –provider="Spatie\Activitylog\ActivitylogServiceProvider"-
tag="activitylog-config"

php artisan migrate
```

Uso Básico

Em sua forma básica, podemos adicionar um log manualmente através do método activity() e retirar suas informações através do método estático `all()` como no exemplo:

```
activity()->log("Mensagem do log");

$ultimoLog = Activity::all()->last() //retorna o último log registrado.;
$ultimoLog->description; //retorna a mensagem: "Mensagem do log"
```

Setando seus atributos

Logs possuem alguns atributos interessantes que podemos utilizar para especificar a ocasião de uma atividade que ocorreu no sistema.

Descrição e Data de Criação

Como mostrado no primeiro exemplo, é possível definir o atributo de descrição para um log, este é muito importante dentre os atributos básicos pois clarifica a ação representada pelo log. Também temos a data de criação, que diz quando a ação foi registrada como log.

Subject (objeto observado) e Causer (objeto acionador)

É possível setar o objeto “observado” no log e seu objeto “acionador”, isto é, o objeto que está sendo referenciado pelo log e o objeto que causou o registro do log (realizou alguma ação sobre o objeto observado). Exemplo:

```
$causer = usuario;
```

```php
$subject = produtoAlterado;

activity()
   ->performedOn($subject)
   ->causedBy($causer)
   ->log('Produto foi editado.');

$ultimoLog = Activity::all()->last();
$ultimoLog->subject; //retorna o produto que foi alterado.

$ultimoLog->causer; //retorna o usuário que alterou o produto.

$ultimoLog->description; //retorna a mensagem: "Produto foi editado."
```

Customizados: Propriedades, Evento e Tap

O pacote também disponibiliza atributos que podem ser customizados da forma a qual o usuário deseja.

As **propriedades** são dados quaisquer que o usuário possa achar relevante para aquele especifico e que podem ser mudados frequentemente, armazenado como um a*rray associativo*.

Os **eventos** são flags, podem ser tidos como uma forma de categorizar logs semelhantes de acordo com o tipo de ação realizada, armazenados como uma s*tring*.

O **tap** é a maneira que temos de alterar dados mais internos do log antes de ser efetivado e salvo. Aqui podemos dar valores para as propriedades criadas e, mais importante, é possível criar atributos totalmente novos para seu log especifico. É único pois o pacote o armazena como uma função, ou seja, pode-se executar qualquer comando antes de criar o dito log.

Acesso

São disponibilizados alguns métodos estáticos para que se possa adquirir logs já registrados pela aplicação, os mais notáveis são `all()`, `where()` e `last()`. Estes pegam todos logs, apenas os logs que correspondem a uma condição e o log mais novo, respectivamente.

Limpeza

Assim como podemos acessá-los, é possível limpar os logs que já existem. Isso é feito pelo prompt utilizando o comando `php artisan activitylog:clean`, que deleta todos os logs mais velhos que o valor presente na variável `clean_after_days` do arquivo de configuração.

Podemos encontrar também dois parâmetros relevantes: `--force` e `--days`. O primeiro pula a confirmação de deleção e o segundo ignora a variável `clean_after_days` e utiliza o valor fornecido como parâmetro.

Uso Avançado

O que foi visto anteriormente é suficiente para utilizar este pacote, porém há maneiras mais otimizadas de faze-lo. No uso avançado decidimos abordar a superfície da criação automática de logs, a leitura de vários logs e como desabilitar o registro de logs em situações específicas.

Tópicos Abordados

1. Instalação
2. Configuração
3. Uso Básico
    1. `Log Manual pelo activity()`
    2. `Atributos de logs`
    3. Acessar logs
    4. Limpar logs
4. Uso Avançado
    1. Log Automático por Logging Model Events
    2. Placeholders e acesso de vários logs.

Desabilitando Logs

Manual:

1. Visualização.
2. Login e Logout.

Acessar:

1. Ultimo login do usuário.

Automático:

Desabilitando Logs:

1. Seeders.